

Intro

For Milestone 2 of our face-verification pipeline, we established an iterative process for evaluating the performance of our verification system. We used this process to run evaluations of the system's baseline configurations that were established in Milestone 1 as well as evaluations of the data-centric change we made to the system for Milestone 2. This report aims to describe this evaluation process, how we made changes to the input data to improve the system from our baseline, and to analyze the errors our system made during evaluations through looking for patterns within false negative and false positive pairs. The ROC curves, confusion matrices, and examples from our error analysis slices are all pictured at the bottom of the report under the "Image and Plot Appendix Section".

Baseline Setup, Tracked Run Process, and Threshold Selection Rule

The dataset of LFW images has been deterministically grouped into pairs, each with a label indicating whether the paired images are of the same person (labeled positive) or of different people (labeled negative). The images have been paired so that 50% of pairs are positives and the other 50% of them are negatives. Each of these pairs have been separated into one of three splits: a training split (70% of the full dataset), a validation split (15% of the dataset), and a test split (15% of the dataset). Of all the 13233 images from the dataset, we chose to randomly select 5000 pairs of images in total to use for the training, validation, and testing of our face verification system. The images for each pair and split were chosen randomly with a fixed seed to ensure their reproducibility.

Within the full dataset, the number of images of each person are varied with some people only having 1 or 2 images and other people having hundreds of images. For the baseline evaluation process, the maximum number of images of each person that were included among all the pairs and splits was set to 999, a number higher than the highest number of images of one individual in the dataset.

For our evaluation process, we first ran the generated validation split through a threshold sweep, testing a range of decision thresholds to obtain and log the identity verification system's performance when set to each threshold. There were 200 threshold values tested, all evenly spaced decimal values from 0.0 to 1.0. All of the pairs in each split along with the other configurations were fixed during this time. The system's performance was measured by calculating these evaluation metrics: accuracy, balanced accuracy, true-positive rate, false-positive rate, F1 score, and the number of true positives, false positives, true negatives, and false negatives at each threshold. The best performing threshold was determined by maximizing the balanced accuracy. The threshold with the maximum balanced accuracy was reported at the end of the threshold sweep along with the evaluation metrics themselves. The evaluation metrics obtained for each threshold were recorded in the `sweep_results.json` file while the best performing threshold, its evaluation metrics and the timestamp of run was recorded in the `runs.json` file. An ROC curve of the run with the best threshold was also generated and stored in the `outputs` folder.

The next part of the evaluation process involved manually adjusting the decision threshold value in the configuration file, eval.yml, to the best reported value from the threshold sweep and running a script that would measure, report, and log the evaluation metrics of both the validation split and the test split with that threshold. Like with the threshold sweep, both of these runs, their evaluation metrics, and their timestamps were recorded in the runs.json file. Confusion matrices for both the evaluation of the validation split and the evaluation of the test split were also generated and stored in the outputs folder.

Data-centric change - Before vs After

To attempt to improve the balanced accuracy of the verification system, we adjusted the pairing of each image so that one individual within the full dataset could only have a relatively small maximum of images of them. In the baseline, when the maximum images per individual was set to 999, there was virtually no limit to images per individual due to the set maximum being set to an arbitrarily high number. In the improved version of the data splits, the maximum images per individual was set to 4 to balance individuals with many images with individuals with very few images, and prevent the domination of those with very little images in the data splits.

Max Images per Individual	Selected Threshold	Threshold Sweep Balanced Accuracy	Val Split Balanced Accuracy	Test Split Balanced Accuracy
999	0.9246	0.6180	0.6190	0.6080
4	0.9196	0.5910	0.5910	0.6080

Contrary to our expectations, when we lowered the limit of the total images per individual that could be included in all pairs, the balanced accuracy obtained from the validation split evaluation was slightly lower than the balanced accuracy obtained with an images per individual limit of 999. However, for both values used for the maximum images per individual, the balanced accuracies were equal when evaluating the test split. In addition to this, the lower limit of 4 marginally increased the system's true positive rate, which can be seen in the confusion matrices in Figures 3 and 4. This leads us to believe that the root of our verification system's error rate does not significantly stem from the fact that some individuals with images in the dataset have significantly more images than other individuals.

Error Slices

To investigate the root of our system's errors, we took the pairs that the system falsely labeled positive (the false positives) and the pairs it falsely labeled negative (the false negatives) and analyzed the images of 20 pairs of each, 10 from the validation split and 10 from the test split.

The pairs analyzed are from the model used after our data-centric change. After running the `run_evaluation` script, they will be stored in the `outputs` directory, within the `false_positives` and `false_negatives` directories respectively. Example image pairs that were in the false negative and false positives slices can be found at the bottom of the report in Figures 5 and 6.

After analyzing the set of false negative pairs, we noticed in each of the images there were noticeable differences even though each image in the pair contained the same person. While they were all close-up shots of the faces, the images captured them in different lighting, making different facial expressions, or looking in different angles. Some pairs even contained an image of a person's face partially obstructed by an object while in the other image, the same person's face is completely visible. Since our system solely measures the cosine similarity of each pixel in the paired images, it is likely that our system incorrectly regards these differences in expression, lighting, and angle within same-identity image pairs as signs that the paired images are of different people.

The images within the false positive pairs had less consistent patterns, but a notable pattern was the likelihood of images within each pair containing different people with the same smile lines, and these smile lines were usually well-defined and obvious creases in the face. Other patterns shared between these images were similar colors of lips, similar angles of faces, similar facial expressions, or similar eye shapes. There was even one positively-labeled pair that contained one image of a person with prominent pink lips and an image of a different person with his tongue partially out, obscuring his lips. Like with the set of false negative pairs, our system seems to have labeled the false positives based on the overall similarity of the color of each image's pixels, regarding different people whose faces make similar shapes or face the same direction as the same person.

Closing Statement

Through this iteration of evaluation runs and error analysis, we have learned the importance of the properties and quality of input data when developing and training ML systems. Going forward, we plan to make further improvements to our input data to bolster the system's success rate in future milestones.

Image and Plot Appendix:

Figure 1: Baseline ROC Curve

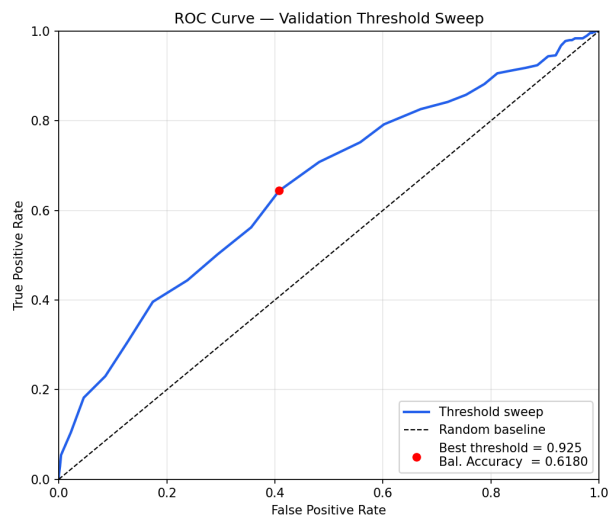


Figure 2: ROC Curve after Data-Centric Change

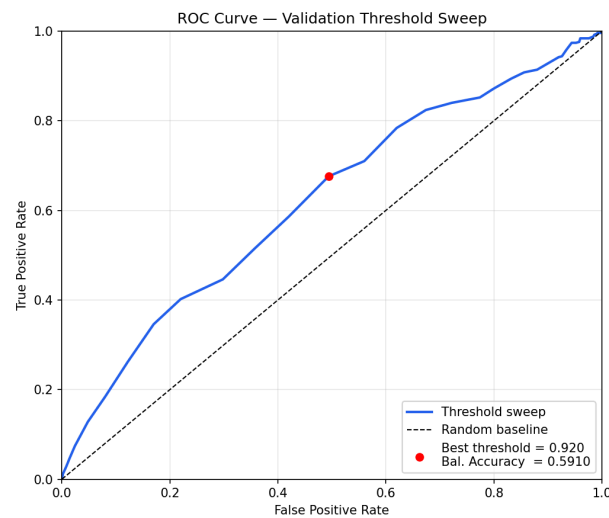


Figure 3: Baseline Confusion Matrices (Left: Validation Split; Right: Test Split)

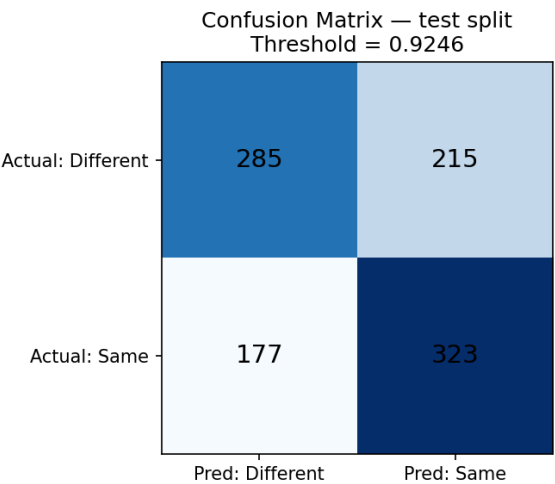
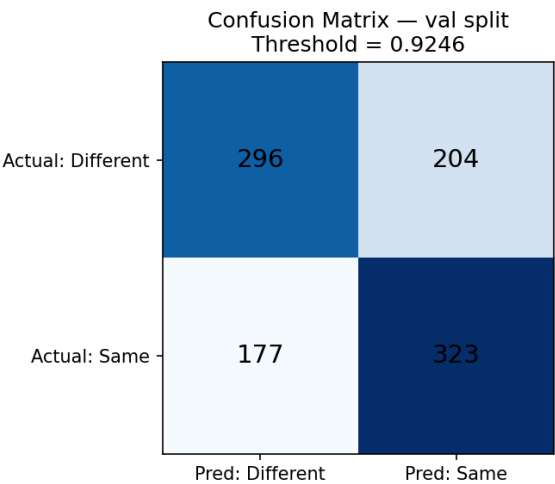


Figure 4: Confusion Matrices After Data-Centric Change (Left: Validation Split; Right: Test Split)

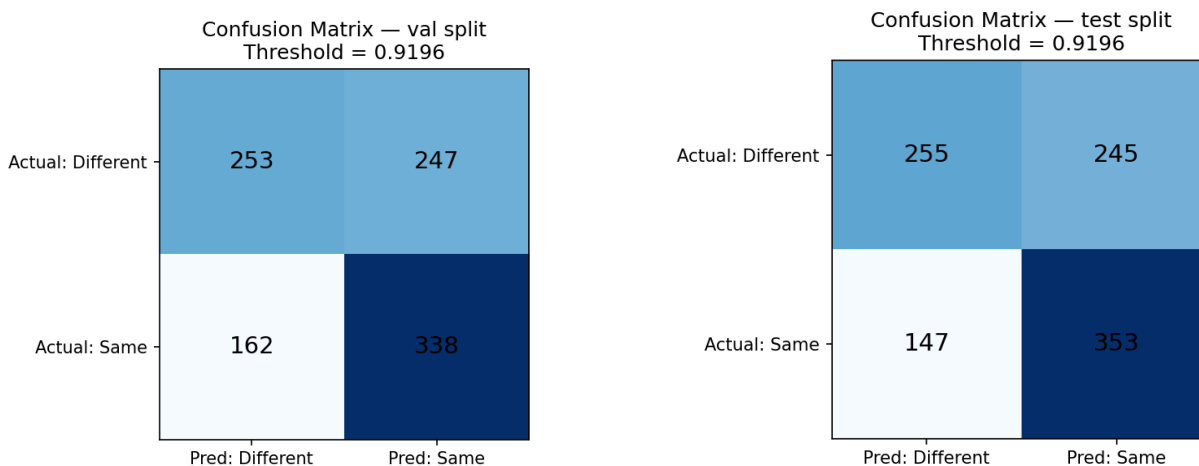


Figure 5: Examples in False Negative Slice



Figure 6: Examples in False Positive Slice

